LIBXSTREAM Samples

DBM — OpenCL

OpenCL backend of CP2K's DBM (Distributed Block-sparse Matrices). DBM processes batches of small matrix multiplications (tasks):

```
typedef struct {
  int m;
  int n;
  int k;
  int offset_a;
  int offset_b;
  int offset_c;
} dbm_task_t;
```

Each task is characterized by its M, N, and K and by offsets into compact arrays holding the A, B, and C matrices of an entire batch. A universal kernel covers a general range of shapes, and homogeneous batches are specialized at runtime. The entry point is `dbm_multiply_opencl_launch_kernel` in dbm_opencl.h.

Build

```
cd samples/dbm
make
```

Produces `dbm_bench.x`, which is the benchmark driver of the SMM sample linked with the DBM backend. The SMM objects are built by the SMM sample, hence its make variables (`WITH_GPU`, `ELEM_TYPE`) apply. Requires an OpenCL runtime, BLAS, and LIBXS built from a sibling directory.

Test

```
make test
```

Runs `dbm_bench.x` with `DBM_MULTIPLY_SMM=-1`, which routes the homogeneous LIBSMM batches into the DBM kernel and validates the result. The test fails if the DBM kernel did not run. The arguments of `dbm_bench.x` are those of `acc_bench.x` and can be given as `make test TESTARG="5 3000 13x5x7"`.

CP2K

CP2K builds the backend into its own targets without naming its files: CMake calls `libxstream_add_dbm(<target>)` after `find_package(libxstream)`, and the Makefile in CP2K's `src/dbm` includes `dbm.mk` from this directory. Both generate `dbm_kernels.h` into the consumer's build tree. To build CP2K's DBM miniapp against this directory:

```
make miniapp CP2K_ROOT=/path/to/cp2k
```

`CP2K_ROOT` is optional and defaults to a `cp2k` directory next to LIBXSTREAM, and `dbm_miniapp.x` is built in `$CP2K_ROOT/src/dbm`. Unlike `make test`, the miniapp also exercises heterogeneous batches. `SMM=1` links LIBSMM from the SMM sample, which `DBM_MULTIPLY_SMM` needs.

Environment Variables

Variable	Default	Description
DBM_MULTIPLY_SMM	0	Positive: homogeneous batches with M, N, and K up to this value use LIBSMM (1 = 64, at most 127). Negative: LIBSMM (DBCSR) uses the DBM kernel for homogeneous batches (backward usage, experimental)
DBM_MULTIPLY_SORT	0	Order of tasks in a batch: 0 as generated, 1 by shape
DBM_MULTIPLY_KERNEL	-	Path to a `.cl` file that replaces the embedded kernel; the embedded kernel is used if the file cannot be opened
DBM_MULTIPLY_FP	0	1: compute in single precision (data remains double precision)
DBM_MULTIPLY_BN	auto	Column tile (1–32): 8 (2 on NVIDIA), or that times twice SM if SM is non-zero
DBM_MULTIPLY_BK	0	Caps the K-block of mixed shapes (1, 4, 8, or 16 derived from the batch's max K); homogeneous batches use their exact K
DBM_MULTIPLY_SM	0	Non-zero: accumulate in shared local memory
DBM_MULTIPLY_WG	auto	Work-group size
DBM_MULTIPLY_LU		

| 0 | Unrolling: -2 full, -1 no hints, 0 inner, 1 outer-dehint | | DBM_MULTIPLY_RO | -1 | Read-only operands: negative = auto, 0 = global, 1 = constant | | DBM_MULTIPLY_XF | -1 | Intel 256-GRF mode: negative = device default, 0 = off, 1 = on | | DBM_MULTIPLY_NZ | 0 | Non-zero: skip zero contributions when accumulating into C | | DBM_MULTIPLY_LIN | 0 | Non-zero: swap the access pattern of A and B | | DBM_MULTIPLY_SGB | auto | Sub-group broadcast: on by default on Intel GPUs (0 = off), opt-in on NVIDIA (1) | | DBM_MULTIPLY_BLK | auto | Row per lane (1, needs LIN=0) for mixed shapes, on Intel GPUs also for homogeneous batches; 2 adds homogeneous batches elsewhere, 0 = off | | DBM_MULTIPLY_FUSE | auto | Tasks per work-group fusing those sharing A: 16 with BLK, else 1 |

`LIBXSTREAM_VERBOSE=2` (or higher) prints the kernel configuration, the compilation of each specialization, and a summary at exit; values above 2 or negative values also trace every launch.

Mixes — OpenCL

Kernels that are each a known mixture of instructions: a STREAM triad (`mixes_stream`), a GEMM tiled in local memory (`mixes_gemm`), FP atomics (`mixes_atomics`, with the expression `libxstream_opencl_flags_atomics` selects for the device), small blocks gathered by index and multiplied in local memory (`mixes_gather`), and an empty kernel over as many work-items as the stream (`mixes_empty`), which is what dispatching them costs. Timed on a device, they say what each mixture costs there. Counted by a simulator, they are what another kernel's instruction counts are expressed in (tool_score.py).

Build

```
cd samples/mixes
make
```

Produces `mixes.x`. `ELEM_TYPE=float` builds single precision (default: `double`). Requires an OpenCL runtime and LIBXS in a sibling directory.

Run

```
./mixes.x [stream-elements [gemm-order [atomic-slots [launches]]]]
```

Defaults: 1048576 elements, order 256 (rounded up to the tile of 16), 1024 slots (the atomics accumulate the stream elements into them), and 3 timed launches after one warm-up. The gather covers as many 16x16 blocks as the stream has elements. Every result is checked; a mismatch fails the run.

Ozaki Scheme — OpenCL

High-precision GEMM on OpenCL devices via mantissa slicing (Scheme 1) or Chinese Remainder Theorem (Scheme 2). Both schemes decompose FP matrices into int8/u8 tiles and use the DPAS/XMX matrix engines (Intel) or dp4a, `mma.sync` and warp-group MMA (NVIDIA) when available; Scheme 2 can also carry its residues in bf16 (`OZAKI_BF16`). This is an OpenCL adaptation of the CPU-based Ozaki sample in LIBXS.

Build

```
cd samples/ozaki
make [DBG=1]
```

Requires an OpenCL runtime and headers, a LIBXS checkout next to this repository (`../libxs`, which provides `Makefile.inc`), and a host BLAS for the reference GEMM (linked via `BLAS=2`). `make test` runs `./ozaki.x 64 64 64`. Compile-time defines go into ECFLAGS, e.g. `make ECFLAGS="-DOZAKI_CUBLAS_WORKSPACE=0"`.

Run

```
./ozaki.x [M [N [K [transa [transb [alpha [beta [lda [ldb [ldc]]]]]]]]]]
```

All arguments are positional and optional:

| Pos. | Argument | Default | Description | ||||| | 1 | M | 257 | Rows of C and op(A) | | 2 | N | M | Columns of C and op(B) | | 3 | K | M | Inner dimension | | 4 | transa | 0 | 0=N, 1=T for A | | 5 | transb | 0 | 0=N, 1=T for B | | 6 | alpha | 1 | Scalar multiplier for A*B | | 7 | beta | 1 | Scalar multiplier for C | | 8 | lda | auto | Leading dimension of A | | 9 | ldb | auto | Leading dimension of B | | 10 | ldc | M | Leading dimension of C |

```
./ozaki.x 256                       # adaptive scheme (default)
OZAKI=2 ./ozaki.x 4096              # Scheme 2
OZAKI_CACHE=3 ./ozaki.x 4096        # adaptive, with preprocessing cache
```

Environment Variables

Scheme Selection | Variable | Default | Description | |||| | OZAKI | 3 | 1=mantissa slicing, 2=CRT, 3=adaptive | | OZAKI_FP | 64 | 64=fp64 (double), 32=fp32 (float) | | OZAKI_N | (auto) | Slices (Sch.1: fp64=8, fp32=4) or moduli (Sch.2: 2–20, see below). Under OZAKI=3 it sets the moduli only | | OZAKI_XOVER | 27 | Sch.1/2 crossover weight used by OZAKI=3 (fp64, non-NVIDIA) |

The default modulus count is the smallest that carries the exact product for K up to `OZAKI_MAXK`: 16 (fp64) and 9 (fp32) at the default `OZAKI_MAXK`, more for a larger one.

OZAKI=3 picks a scheme per call from that call alone, so mixed matrix sizes in one process do not influence each other. On NVIDIA it selects Scheme 2, and in fp32 on other devices Scheme 1.

Accuracy | Variable | Default | Description | |||| | OZAKI_FLAGS | 3 | Sch.1 bitmask: 1=Triangular, 2=Symmetrize, 0=full S^2. No Sch.2 (OZAKI=2 defaults to 0) | | OZAKI_TRIM | 0 | Levels to trim (0=default). ~7 bits (Sch.1), ~4 bits (Sch.2); negative buys precision back | | OZAKI_MAXK | 32768 | Largest K one preprocessing pass covers; sizes the moduli and the K-grouping. 0 or negative=default | | OZAKI_FRACCRT | (auto) | Sch.2: 0=Garner, 1=flat fractional (not exact), 2=fractional CRT. Auto: 0 if unfused, on NV MMA or without fp64, else 2 | | OZAKI_TZDETECT | 0 | Sch.2: report the largest `OZAKI_TRIM` that stays exact for the data (reporting only) |

Under OZAKI=3, a positive `OZAKI_TRIM` applies to Scheme 1 only; a negative one applies to both schemes. There is no unbounded `OZAKI_MAXK`.

Only `OZAKI_FLAGS` 0 and 3 compute the full product; 1 and 2 are for symmetric operands only. The LIBXS interceptor accepts only 0 and 3.

`OZAKI_FRACCRT` 0 and 2 are exact; 1 is magnitude-bounded. Without fp64 only Garner (0) is available.

`OZAKI_TZDETECT=1` prints per call how many moduli the data needs and the matching `OZAKI_TRIM`; it adds a device-to-host readback per call.

Scheme 2 (CRT) | Variable | Default | Description | |||| | OZAKI_HIER | (auto) | Hierarchical (two-level) CRT reconstruction (default on) | | OZAKI_HIER_L2 | (auto) | Level-2 combine: 1=tree merge, 0=Garner. Auto: tree merge for at most 2 groups (more fall back to Garner) | | OZAKI_L1_GARNER | 0 | 1=Garner at level 1 instead of the explicit CRT (not with OZAKI_FRACCRT=2) | | OZAKI_EXTRACT_FLAT | 0 | fp64: 1=extract the residues per modulus instead of per group | | OZAKI_MULHI | (auto) | 64-bit `mul_hi` at level 2: 0=built-in, 1=from 32-bit halves. Auto: probed (see below) | | OZAKI_UNFUSE | (auto) | Reconstruct in a 2nd kernel (`gemm_crt_reduce`). On for GPUs | | OZAKI_GROUPS | (auto) | K-grouping factor (1=off). Consecutive K panels share the reconstruction | | OZAKI_SYMRES | (auto) | Symmetric residues (magnitude m/2 at most). On with bf16, or `OZAKI_MAXK` past 32768 | | OZAKI_PB | 1 | CRT modulus batching factor | | OZAKI_BF16 | 0 | Carry the residues in bf16 rather than int8 (see below) | | OZAKI_COMPLEX_3M | (auto) | Complex: 1=3M on residues, 0=4M; auto: 3M from _MIN on. Needs OZAKI_UNFUSE, not with OZAKI_BF16/OZAKI_TZDETECT | | OZAKI_COMPLEX_3M_MIN | (auto) | Cube root of M*N*K from which auto takes 3M (1024 coherent, 4096 discrete) | | OZAKI_SWIZZLE | 0 | Work-group rasterization width (0=launch order) | | OZAKI_NOBOUNDS | 0 | Drop output range checks (whole-tile shapes only) | | OZAKI_ALPHA_ONE | 0 | Specialize the GEMM for alpha=1 | | OZAKI_FIRST | 0 | Specialize the GEMM for

C=0+AB (first accumulation) | | OZAKI_SKIP_GARNER | 0 | Fused epilogue only (OZAKI_UNFUSE=0): skip reconstruction (timing only, result is not a GEMM) |

`OZAKI_UNFUSE` gives the same results as the fused epilogue (`OZAKI_UNFUSE=0`). It needs `OZAKI_HIER`, `OZAKI_PB=1` and no K-grouping, and one byte of scratch per modulus and output element, twice the size of C in fp64 at 16 moduli. In the `LIBXSTREAM_PROFILE` report the two kernel rows add up to the total; the FLOP rate is attributed to the GEMM row.

`OZAKI_GROUPS` disables the unfused epilogue and is slower for it (reported under `OZAKI_VERBOSE`); set it only when the per-pass buffers do not fit. Left unset, it is derived only where an exact result needs it: an `OZAKI_MAXK` past what an integer accumulator holds exactly (32768 unsigned, 131072 symmetric). `OZAKI_SYMRES` turns on by itself past 32768, so the grouping is derived only past 131072 (or past 32768 with `OZAKI_SYMRES=0`). The bf16 carrier never needs it. `OZAKI_GROUPS=1` declines the derived grouping and warns where that leaves K unreduced; `OZAKI_GROUPS=0` means unset.

`OZAKI_MULHI` is settled at initialization by running a small kernel: a device compiler may get the 64-bit `mul_hi` wrong for a constant operand, and every hierarchical reconstruction is then wrong. Setting it skips the probe; 0 on such a device gives wrong results.

Flat reconstruction takes `OZAKI_HIER=0 OZAKI_FRACCRT=0` with `OZAKI=2`; otherwise `OZAKI_HIER=0` warns and the reconstruction stays hierarchical.

`OZAKI_BF16=1` runs Scheme 2 on a floating-point matrix engine, for a device whose bf16 matrix throughput is much higher than its integer throughput. Moduli, accuracy and results are unchanged (bit-identical to int8). It needs warp-group MMA with the default A and B layouts, or warp-level MMA (`mma.sync`), where it selects `OZAKI_BKMAJOR` by itself; otherwise it disables itself with a message.

The complex variables take effect through the LIBXS interceptor (ZGEMM/CGEMM); `ozaki.x` issues real GEMMs only.

NVIDIA | Variable | Default | Description | |||| | OZAKI_WGMMA | (auto) | Sch.2: warp-group MMA. On where reachable (see below) | | OZAKI_WGMMA_N | 256 | Warp-group tile width, 64, 128 or 256 | | OZAKI_WGMMA_M | 128 | Warp-group tile rows: 64 = one warp group, 128 = two, 256 = four (with OZAKI_WGMMA_N of 128 or less) | | OZAKI_WGMMA_KU | 0 | Staging depth per tile, overriding the derived one (0=derived; still capped to the shared-memory budget) | | OZAKI_WGMMA_DEFER | (auto) | Deferred wait: 1=on, 0=off. Auto: on for the 128x256 tile | | OZAKI_WGMMA_STAGES | (auto) | Staging buffers with the deferred wait, 2 to 4. Auto: 4, 2 with OZAKI_BF16 | | OZAKI_WGMMA_RESIDE | (auto) | Sch.2: tiles per SM below which 128x128 runs two per SM (0=off) | | OZAKI_WGMMA_RESIDE_MKT | 2.5E8 | Sch.2: bound on MKtiles per SM for 128x128 (0=no bound) | | OZAKI_MAXNREG | 0 | Sch.2 wgmma: registers per thread of the GEMM (0=derived) | | OZAKI_ABLOCK | 1 | wgmma: A in the fragment layout (0=plain layout; OZAKI_BF16 needs 1) | | OZAKI_BVNNI | 1 | Sch.2: pre-interleave B so each operand is one aligned uint. Not with wgmma or OZAKI_BKMAJOR | | OZAKI_BKMAJOR | 0 | Sch.2 without wgmma: transpose B to [N][K] instead (see below) |

Warp-group MMA (`wgmma`) is the default wherever initialization finds it usable; otherwise `mma.sync` is used and `OZAKI_VERBOSE=1` says so. `OZAKI_WGMMA=0` selects `mma.sync`. It needs `OZAKI_HIER`, no K-grouping, `OZAKI_PB=1` and `OZAKI_FRACCRT` other than 1, and it fixes the tile to `OZAKI_WGMMA_M`x`OZAKI_WGMMA_N` regardless of `OZAKI_TM`/`OZAKI_TN`. The first JIT per shape takes about twice as long; a failed patch refuses the kernel, see `OZAKI_VERBOSE=1`.

Under warp-group MMA, `OZAKI_KU` sets the base staging depth (8 at the default width, 16 with `OZAKI_WGMMA_N` of 64 or 128). The depth per tile is derived from it within 128 KB of shared memory, so lower it if a device refuses that.

Below `OZAKI_WGMMA_RESIDE` tiles per SM the tile narrows to 128x128 and two work-groups share an SM. The default is 8 on a part with a coherent host link and 2 otherwise; `OZAKI_WGMMA_RESIDE=0` keeps the wide tile. The wide tile is also kept once M*K*(tiles per SM) exceeds `OZAKI_WGMMA_RESIDE_MKT`, and `OZAKI_WGMMA_RESIDE_MKT=0` lifts that bound.

`OZAKI_BKMAJOR=1` applies to the `mma.sync` and dp4a paths, not to warp-group MMA, and is exact like the default layout.

`LIBXSTREAM_DUMP=2` (or a negative value) also writes the patched PTX of a warp-group kernel as `<name>.ptx`.

Hardware Control | Variable | Default | Description | |||| | OZAKI_TM | (auto) | Output tile M (BM), with OZAKI_TN. Not under wgmma | | OZAKI_TN | (auto) | Output tile N (BN), with OZAKI_TM. Not under wgmma | | OZAKI_RTM | (auto) | Register tiling M (power of two). Auto: 2 (Intel, NV MMA), 4 (Intel 256-GRF without bit 2 of OZAKI_FLAGS), 1 (other) | | OZAKI_RTN | (auto) | Register tiling N. Auto: 8 (NV MMA Sch.2), 2 (Intel, NVIDIA), 1 (other) | | OZAKI_SB | 1 | Sch.1: slice-block width for the pair loop; must divide the slice count | | OZAKI_WGS_MAX | (auto) | Work-group size ceiling for tile selection (0=hardware bound). Auto: 4 sub-groups on non-Intel GPUs, else 0 | | OZAKI_TILE_SAT | (auto) | Tile selection keeps at least compute units / OZAKI_TILE_SAT work-groups. Auto: 16, 1 on non-Intel GPUs | | OZAKI_KU | 2 | K-loop unroll factor. Auto: 2, 4 (NVIDIA dp4a, Sch.2), 1 (mma.sync, not adjustable); wgmma: see NVIDIA | | OZAKI_RC | 8 | DPAS repeat count (8 or 4) | | OZAKI_BIGGRF | (auto) | Override 256-GRF detection (0=off, 1=on). HIER defaults to 128; LIBXSTREAM_BIGGRF applies if this is unset | | OZAKI_LU | 0 | Loop-unroll hints: -2=all loops, -1=none, 0=inner loops, 1=inner loops and outer loops not unrolled | | OZAKI_PREFETCH | 0 | Sch.1: enable prefetching | | OZAKI_SCALAR_ACC | 0 | Sch.1: force scalar accumulation | | OZAKI_DEFINES | (unset) | Extra build options (e.g. -DNAME=value) for the Scheme-1 and Scheme-2 kernel programs | | OZAKI_WG | 0 | Read but has no effect | | OZAKI_SG | (auto) | Has no effect: the sub-group size is 16, or 32 with NV MMA |

`OZAKI_TM` and `OZAKI_TN` take effect only together and not under warp-group MMA (reported at `OZAKI_VERBOSE=1`); otherwise the output tile is chosen per call from M and N, keeping enough work-groups to fill the compute units (`OZAKI_TILE_SAT`).

On Intel GPUs the register tiling is chosen per call as well: `OZAKI_VERBOSE=3` prints the range (e.g. `rtm=2..4`) and the `JIT` line the value used (`rt=`). Setting `OZAKI_RTM` or `OZAKI_RTN` disables this for both; the one not set keeps its base value.

Kernels are compiled on first use per tile, register tiling and bounds-checking variant (Scheme 1 also per cutoff, Scheme 2 per modulus count) and kept for later calls.

Memory and Caching | Variable | Default | Description | |||| | OZAKI_CACHE | 0 | Preprocessing cache bitmask: 1=A, 2=B, 3=both (or any negative) | | OZAKI_ARENA | (auto) | Device scratch arena in MB (0=off). Auto: on if no pool | | OZAKI_NPANEL | 1 | Sch.2: N-panel width (0=auto, 1=disable). See below |

`OZAKI_CACHE` assumes that the content at a cached pointer does not change between calls; enable it when A or B stays constant. Only `OZAKI_CACHE=0` disables it: a negative value is another spelling of 3. Scheme-1 calls on cached operands also skip the occupancy readback.

`OZAKI_ARENA` keeps the per-call device scratch (operand planes, residue planes, device copy of C) in one allocation for the life of the context. Unset, it is on wherever LIBXSTREAM has no device memory pool. `OZAKI_ARENA=0` restores per-call allocation. A positive value caps the arena in MB and also enables it where a pool exists (e.g. `LIBXSTREAM_USM` on NVIDIA); a call needing more than the cap falls back per buffer, and a first call that exceeds it gets no arena at all. `ozaki_scratch_size` reports the requirement. Planes kept by `OZAKI_CACHE` are never carved from the arena. A caller that owns device memory can supply it with `ozaki_scratch_size`/`ozaki_scratch_set` (the benchmark does so under `OZAKI_SCRATCH=1`); the LIBXS interceptor relies on the default.

`OZAKI_NPANEL=0` (Scheme 2) splits N into panels whose upload, preprocessing, GEMM and download overlap; a value above 1 sets the panel width (rounded up to the tile). Panelling is skipped with too few tiles per panel, `N` not above the tile width, K-grouping, device-resident operands, or a B cache (`OZAKI_CACHE=2` or 3), which takes precedence. With `transb`, B is uploaded whole. The panel slots are `OZAKI_NSLOTS` (compile-time, 2 by default).

Benchmark | Variable | Default | Description | |||| | NREPEAT | 1 | Number of benchmark repetitions | | OZAKI_VERBOSE | 0 | 0=silent, non-zero=notes and warnings, 3+=all, negative=all | | GRADE | 0 | Componentwise grade of every result (non-zero=on) | | EVIL | 0 | Per-column exponent span of 2n binades (n>0 only) | | TAME | 0 | Keep n mantissa bits of A and B, clear the rest | | XREF | 0 | Compensated reference on n rows (negative=all) | | OZAKI_SCRATCH | 0 | Non-zero=supply caller-owned device scratch (ozaki_scratch_size/ozaki_scratch_set) | | OZAKI_SWITCH | (unset) | Switch to n moduli after the warm-up call (2 up to the initial count) |

`XREF=n` grades the Ozaki, the host BLAS and the cuBLAS result against a reference computed by `libxs_dot2_gemm` (as if in twice the double precision) on n rows spread over C, and prints one line `XREF: rows=n ozaki_linf_rel=.. blas_linf_rel=.. cubla`s with `_l2_rel` and, under `GRADE`, `_grade` per result. With `GRADE` and a cuBLAS result, a `vs-cuBLAS DIFF` line also grades the Ozaki result against cuBLAS.

Additional variables for accuracy monitoring and complex GEMM dispatch (`OZAKI_THRESHOLD`, `OZAKI_STAT`, `OZAKI_EPS`, `OZAKI_RSQ`, `OZAKI_EXIT`, `OZAKI_COMPLEX`) are handled by the LIBXS Ozaki sample, which owns the GEMM interceptor. There, `OZAKI`, `OZAKI_N`, `OZAKI_FLAGS`, `OZAKI_TRIM`, `OZAKI_MAXK`, `OZAKI_GROUPS`, `OZAKI_TM`, `OZAKI_TN` and `OZAKI_VERBOSE` are read by

LIBXS and passed on, with the restrictions its README states; the kernel and memory variables above are read by LIBXSTREAM itself in either case.

In `ozaki.x`, the driver reads `OZAKI`, `OZAKI_FP`, `OZAKI_FLAGS`, `OZAKI_TRIM`, `OZAKI_GROUPS` and `OZAKI_VERBOSE` and passes them to `ozaki_init`, which reads the other kernel and memory variables.

Kernel timings come from LIBXSTREAM: `LIBXSTREAM_PROFILE=1` reports one row per kernel, and `LIBXSTREAM_PROFILE_MEM=1` reports transfer rates.

cuBLAS Reference When the OpenCL headers are taken from a CUDA installation, the sample additionally links cuBLAS and reports a device-side reference GEMM (`cuBLAS GEMM`, and `cuBLAS DIFF` against the host BLAS). Build with `make CUBLAS=0` to opt out. `CUBLAS=1` (the default) also forces `PEDANTIC=0`. A failing cuBLAS run is not fatal.

By default the sample requests FP64 emulation without the fallback to native FP64 (CUDA 13.0u2 and later, compute capability 8.0 and later; fp32 emulation needs compute capability 10.0). The variables below are populated before cuBLAS is entered; any of them already set wins:

| Variable | Sample default | ||| | CUBLAS_EMULATE_DOUBLE_PRECISION | 1 | | CUBLAS_EMULATE_SINGLE_PR
| 1 (`OZAKI_FP=32`) | | CUBLAS_EMULATION_STRATEGY | eager | | CUBLAS_EMULATION_SPECIAL_VALUES_SUPP
| 0 | | CUBLAS_FIXEDPOINT_EMULATION_MANTISSA_BIT_COUNT | from `OZAKI_CUBLAS_BITS` (unset if 0) |

| Variable | Default | Description | |||| | OZAKI_CUBLAS_BITS | 0 | Mantissa bits: 0=default, <0=match the OZAKI_N slices | | OZAKI_CUBLAS_XPTR | 0 | 1=pass LIBXSTREAM device pointers to cuBLAS (experiment) |

A non-zero `OZAKI_CUBLAS_BITS` fixes the number of int8 slices (`slices = ceil((bits + 1) / 8)`) and is passed as `CUBLAS_FIXEDPOINT_EMULATION_MANTISSA_BIT_COUNT` unless that is already set. A negative value takes the component count of this sample, which is a slice count for OZAKI=1 and a modulus count otherwise. `OZAKI_CUBLAS_XPTR=1` is expected to fail: LIBXSTREAM device pointers are not addresses of the CUDA context.

Emulation needs a workspace, which the sample supplies (compile-time `OZAKI_CUBLAS_WORKSPACE` in bytes, 2 GB by default, 0 to disable; see Build); a workspace that is too small silently disables emulation.

Small Matrix Multiplication (SMM) — OpenCL

Batched small matrix multiplications (SMM) on OpenCL devices via the ACC LIBSMM interface. Originated from the DBCSR OpenCL backend, adapted to run on top of LIBXSTREAM. Includes GPU-accelerated kernels, a benchmark driver, and an auto-tuning framework.

Build

```
cd samples/smm
make [WITH_GPU=<device>] [ELEM_TYPE=<type>]
```

Produces `acc_bench.x`. Requires an OpenCL runtime, BLAS (`BLAS=2`), and LIBXS in a sibling directory (header-only unless a LIBXS library is built).

| Make variable | Default | Description | |||| | ELEM_TYPE | `double` | Element precision: `double` or `float` | | WITH_GPU | `GPUVER` | Device for tuned parameters (PVC, A100, ...), see below | | SYM | 0 | Non-zero also keeps comments in the embedded kernel source |

`make test` builds and runs `acc_bench.x` with its defaults.

The build embeds `params/tune_multiply_<WITH_GPU>.csv`. Without such a file, it embeds `tune_multiply.csv` from this directory if present (the default output of a merge), and otherwise every `params/*.csv` unless `WITH_GPU` was given on the command line.

A CMake project (e.g., DBCSR) builds LIBSMM into its own target with `libxstream_add_smm(<target>)` after `find_package(libxstream)`, which also generates `smm_kernels.h` into the target's build tree.

A Makefile project (e.g., CP2K's DBM miniapp) includes `smm.mk` instead, optionally setting `LIBXSTREAM_SMM_GENDIR` (directory of the generated `smm_kernels.h`, default: current directory) and `LIBXSTREAM_SMM_GENARG` (e.g., `-d` to keep comments). It then adds `LIBXSTREAM_SMM_IFLAGS` to its compiler flags, `LIBXSTREAM_SMM_OBJS` to its objects, and builds them from `$(LIBXSTREAM_SMM_DIR)/%.c` with `$(LIBXSTREAM_SMM_HEADERS)` as prerequisites.

Benchmark Driver

```
./acc_bench.x [nrepeat [batchsize [M [N [K [nc [na [nb]]]]]]]]
```

The kernel shape can also be given as `MxNxK`:

```
./acc_bench.x 5 30000 13x5x7
```

The first argument can be a file with one set of parameters per line. The batchsize argument accepts K/M/G suffixes for memory-budget mode. A negative batchsize, e.g., -30000, draws a random multiple of 100 up to its magnitude (also via `BATCHSIZE_SMM`).

Argument	Default	Description
nrepeat	3 (CHECK=0: 66)	Number of timed repetitions
batchsize	30000	Matrix products per batch (stack size)
M	23	Rows of A and C
N	M	Columns of B and C
K	M	Columns of A / rows of B
nc	batchsize/16	Number of unique C-matrices
na	10*nc* / *Number of unique A-matrices*	/ *nb* / *10*nc	Number of unique B-matrices

Environment Variables (Benchmark)

Variable	Default	Description
CHECK	1	Tolerance: CHECK x 1E-3 (float: 2E-3), sign ignored, 0=off
CHECK_H2D	-	Minimum H2D bandwidth (GB/s); fail if below
CHECK_DEV	-	Minimum device GFLOPS/s; fail if below
CHECK_HST	-	Minimum host GFLOPS/s; fail if below
DEVICE	0	Device index (multi-rank: device = rank % ndevices)
NREPEAT_H2D	1	H2D copy repetitions for bandwidth measurement
NREPEAT_SMM	1	Repeats the batch inside each SMM kernel (for profiling)
BATCHSIZE_SMM	-	Override batchsize (and optionally nrepeat: `bs,nrep`)
UNIFIED	0	1: kernels read the stack from host memory if possible

LIBSMM Kernel Parameters

OpenCL kernels are generated at runtime for any (M, N, K) shape with M*N <= MAX_KERNEL_DIM^2 (the caller's limit; `acc_bench.x` uses 80). Tuned parameters can be embedded at build time or loaded at runtime for better performance. `LIBXSTREAM_DEVMATCH` (which device's parameters are taken) and `LIBXSTREAM_VERBOSE` (INFO output about loaded parameters and generated kernels) are described on the LIBXSTREAM OpenCL page.

Environment Variables (LIBSMM) Transpose kernel:

Variable	Default	Description
OPENCL_LIBSMM_TRANS_BUILDOPTS	-	Extra OpenCL build options
OPENCL_LIBSMM_TRANS_INPLACE	0	Non-zero: in-place if square (no LDS)
OPENCL_LIBSMM_TRANS_BM	auto	Block size in M-direction (0 < BM <= M)
OPENCL_LIBSMM_TRANS_BS	8	Matrices per work-group
OPENCL_LIBSMM_TRANS_PARAMS	-	0: ignore TRANS_BM and TRANS_BS
OPENCL_LIBSMM_TRANS_PREDICT	-	Positive: ignore TRANS_BM and TRANS_BS

Multiply kernel:

Variable	Default	Description
OPENCL_LIBSMM_SMM_BUILDOPTS	-	Extra OpenCL build options
OPENCL_LIBSMM_SMM_KERNEL	-	OpenCL source file replacing the embedded multiply kernel
OPENCL_LIBSMM_SMM_PARAMS	auto	CSV/model file, a parameter row, or 0 for plain defaults (see below)
OPENCL_LIBSMM_SMM_PREDICT	0	Model: -1=off (default if PARAMS=0), 0=untuned shapes, 1=all shapes
OPENCL_LIBSMM_SMM_PREDICT_CONF	0	Confidence a predicted parameter needs; below, its default is used
OPENCL_LIBSMM_SMM_DEVID	-	Device ID whose parameters are used, e.g., `0x0bd5`
OPENCL_LIBSMM_SMM_S	64	BS > 1 needs a batch of at least BSxS; negative: always BS=1
OPENCL_LIBSMM_SMM_BS	auto	Intra-kernel mini-batchsize
OPENCL_LIBSMM_SMM_BM	auto	Block size in M-direction (0 < BM <= M), ignored if LU > 1
OPENCL_LIBSMM_SMM_BN	auto	Block size in N-direction (0 < BN <= N)
OPENCL_LIBSMM_SMM_BK	auto	Block size BK (0 < BK <= M)
OPENCL_LIBSMM_SMM_WS	max(M,N)	Minimum work-group size (WS <= M*N)
OPENCL_LIBSMM_SMM_WG	auto	Work-group size: 0=exact, non-zero=sub-group multiple, 2=power of 2
OPENCL_LIBSMM_SMM_LU	auto	Unrolling: -2=full, -1=no hints, 0=inner, 1=outer, >1=LU blocks of M
OPENCL_LIBSMM_SMM_NZ	0	1: skip atomic updates of zero values
OPENCL_LIBSMM_SMM_AL	0	Ignored in this build
OPENCL_LIBSMM_SMM_TB	0	1: load B (AB=2) only when it changes within a mini-batch
OPENCL_LIBSMM_SMM_TC	1	1: write C back only when it changes
OPENCL_LIBSMM_SMM_AP	0	Parameter stack: 0=global, 1=shared
OPENCL_LIBSMM_SMM_AA	auto	A-matrix: 0=global, 1=shared, 2=register
OPENCL_LIBSMM_SMM_AB	auto	B-matrix: 0=global, 1=shared, 2=register

OPENCL_LIBSMM_SMM_AC | 0 | C-matrix: 0=register, 1=shared | | OPENCL_LIBSMM_SMM_XF | 0 | 1: 256 GRF per thread on Intel GPUs (see below) |

Defaults apply where no tuned or predicted parameter exists. `OPENCL_LIBSMM_SMM_XF` defaults to 1 if `OPENCL_LIBSMM_SMM_BUILDOPTS` contains `-cl-intel-256-GRF-per-thread`. A value of `OPENCL_LIBSMM_SMM_PREDICT` above 1 also selects a LIBXS prediction mode.

`OPENCL_LIBSMM_SMM_PARAMS=0` runs with plain defaults: no tuned table, no prediction (unless `OPENCL_LIBSMM_SMM_PREDICT` is set), and the two-letter variables above (`OPENCL_LIBSMM_SMM_BS` to `_XF`) are ignored, as they are with a positive `OPENCL_LIBSMM_SMM_PREDICT`. In the default build, whose tuned parameters come from the models, `OPENCL_LIBSMM_SMM_PREDICT=-1` alone already drops all tuned parameters.

`OPENCL_LIBSMM_SMM_PARAMS` given a CSV file applies its parameters to any device (device matching is off). Instead of a file, it also takes one parameter row: the CSV columns without DEVICE, S, and GFLOPS, e.g., `3;23;23;23;8;23;1;1;23;0;-1;0;0;0;1;0;2;0;0` (TYPEID;M;N;K;BS;...;AC).

`OPENCL_LIBSMM_SMM_PARAMS` given a saved prediction model (a file written by `predict_params.x`, e.g., trained on part of a table) uses it in place of the embedded models. The embedded tuned parameters still apply unless `OPENCL_LIBSMM_SMM_PREDICT` is positive. `OPENCL_LIBSMM_SMM_PREDICT_CONF` is compared with a forest model's calibrated probability, and with the confidence of any other model.

The full list of tunable parameters is available via `tune_multiply.py --help`.

Tuned Parameters

Pre-tuned parameter sets in `params/`:

| File | Device | ||| | tune_multiply_PVC.csv | Intel Data Center GPU Max (PVC) | | tune_multiply_BMG.csv | Intel Battlemage (BMG) | | tune_multiply_A100.csv | NVIDIA A100 | | tune_multiply_H100-PCI.csv | NVIDIA H100 PCIe | | tune_multiply_H100-SXM.csv | NVIDIA H100 SXM | | tune_multiply_GH200.csv | NVIDIA GH200 | | tune_multiply_V100.csv | NVIDIA V100 | | tune_multiply_P100.csv | NVIDIA P100 | | tune_multiply_Mi250.csv | AMD MI250 |

Parameters are matched by device ID at runtime with best-match fallback. The CSV format takes single and double precision in the same file, and the shipped files hold double precision only.

Each CSV file comes with a prediction model of the same name (`.bin`), built from its double-precision entries; predictions apply to double precision only, other types use the defaults. If every CSV file of a build has a model, the models are embedded instead of the CSV rows. `smm_params.sh` rebuilds the model after each merge, using `predict_params.x` from LIBXS (`samples/predict`, built on demand) found via `LIBXSROOT`, a sibling `libxs`, or `$HOME/libxs`. Without LIBXS, the merge reports the model as stale. To rebuild a model by hand:

```
predict_params.x rf trees8 params/tune_multiply_PVC.csv params/tune_multiply_PVC.bin
```

```
## Use embedded parameters (default)
./acc_bench.x 5 30000 13 5 7

## Plain defaults
OPENCL_LIBSMM_SMM_PARAMS=0 ./acc_bench.x 5 30000 13 5 7

## Load from a specific CSV file
OPENCL_LIBSMM_SMM_PARAMS=params/tune_multiply_PVC.csv ./acc_bench.x 5 30000 13 5 7
```

Rebuild with a specific device's parameters embedded:

```
make realclean
make WITH_GPU=PVC
```

Show which embedded parameter set a device name selects:

```
make match
./acc_match.x "NVIDIA H200"
```

Names given after the device are compared instead of the embedded ones:

```
./acc_match.x "NVIDIA H200" "NVIDIA H100 PCIe" "NVIDIA GH200 480GB"
```

Auto Tuning

Uses OpenTuner to explore the parameter space per (M, N, K) kernel.

```
cd samples/smm
pip install -r requirements.txt
```

Build `acc_bench.x` before tuning. Keep GPU clocks and driver state as stable as practical during a run.

tune_multiply.py Use `tune_multiply.py` when you want direct control over one tuning run or over JSON maintenance.

Tune one kernel and write JSON results in the current directory:

```
./tune_multiply.py 13x5x7
```

Limit the search time, choose the JSON directory, and set the benchmark batch size:

```
mkdir -p params/local
./tune_multiply.py 13x5x7 --stop-after=300 -p params/local -s 30000
```

Tune several explicit kernels from a file, one `MxNxK` per line. A `#` starts a comment, so a line can be annotated or taken out of the list:

```
printf '%s\n' 13x5x7 23x23x23 32x32x32 > kernels.txt
./tune_multiply.py kernels.txt --stop-after=180 -p params/local
```

Merge JSON files into a CSV file (without a shape, `-p` or `--prefer` alone merges as well):

```
./tune_multiply.py -m -p params/local -o tune_multiply_local.csv
```

`-m` alone (or a merge without a shape) merges the precision that `acc_bench.x` was built for and runs it to find out, which needs a device. `-m2` (double), `-m1` (single), and `-m0` (all) merge without one.

A JSON file is named after its kernel and a hash of its parameters, for example `tune_multiply-double-23x23x23-1a2b3c4d.json`, with `-s<N>` after the shape if the batch size is not the default 30000 (`N` is the base-2 logarithm of the batch size, rounded up). Keep one directory per device: a same-named file from another device is reported and left alone.

Update the device name stored in every JSON file of a directory, either as given or, without a name, as the device reports it (which requires running on the target machine):

```
./tune_multiply.py -u "Some Device [0x1234]" -p params/local
./tune_multiply.py -u -p params/local
```

Write the JSON files a CSV file was merged from (the inverse of `-m`), which needs neither a device nor the benchmark:

```
./tune_multiply.py -M params/tune_multiply_PVC.csv -p tmp/PVC
```

A file that exists already is left unchanged.

Check existing JSONs without re-tuning:

```
./tune_multiply.py -c -p params/local
```

When two JSON files describe the same kernel on the same device, `--prefer` decides which one wins the CSV: `fast` (default) keeps the higher GFLOPS/s, and `new` keeps the more recent measurement (e.g., after a driver change). A merge lists the losing files (`Remove N (use -d)`), and `-d` removes them:

```
./tune_multiply.py -p params/local --prefer new
./tune_multiply.py -p params/local --prefer new -d
```

Useful options:

| Option | Meaning | ||| | –stop-after N | Stop the search after N seconds | | -p path | JSON directory; without a shape, merge it | | -s size | Benchmark batch size, also called stack size | | -a level | Tuning level: 0=all ... 4=least tunables | | -m [mode] | Merge JSON files into a CSV file (see above) | | -M file | Write the JSON files of a CSV file (see -p) | | -o file | CSV output file | | -u [device] | Update JSON device names (see above) | | -c [epsilon] | Validate JSON entries | | –prefer fast|new | Duplicate winner; without a shape, merge | | -d | Delete losing duplicates during merge | | -x | Write GFLOPS as 0 into the CSV file | | -e char | CSV separator (default: ;) | | -r count | Repetitions per benchmark run (0: default) | | -mb size | Largest mini-batch size to try (default: 64) | | –timeout seconds | Limit per benchmark run (0: none) | | -q | Skip configurations with AA=1 or AB=1 | | -v | Print the settings of each run |

Setting `OPENCL_LIBSMM_SMM_<X>` (e.g., `OPENCL_LIBSMM_SMM_BS=8`) fixes that parameter during tuning. A value of `tune`, `enabled`, or `on` keeps it tunable unless the tuning level fixes it. `OPENCL_LIBSMM_SMM_WS` is fixed unless set to `tune`.

If a shared LIBXS library loads (`LIBXS_LIB` names the file or its directory, otherwise `LIBXSROOT/lib`, a sibling `libxs/lib`, or the system path), the OpenTuner technique `LibxsBanditA` becomes available (`--technique=LibxsBanditA`). `LIBXS_SURROGATE_KAPPA` (1.0), `LIBXS_SURROGATE_CANDIDATES` (512), `LIBXS_SURROGATE_WARMUP` (8), and `LIBXS_SURROGATE_NOVELTY` (1.0) adjust it.

The tuner can run under MPI. It detects local MPI rank variables and uses them to select `LIBXSTREAM_DEVICE`, so ranks on the same node can use different devices. This is most useful when each rank is tuning a different kernel.

```
mpirun \
  ./tune_multiply.py 13x5x7 --stop-after=300 -p params/local : \
  ./tune_multiply.py 23x23x23 --stop-after=300 -p params/local
```

Interrupted tuning writes the best result seen so far. Unless `--database` names one, the OpenTuner database is kept per process in the system's temporary directory (`opentuner/<pid>.db`, under MPI `opentuner<device>/<pid>.db`), so sessions do not share it.

tune_multiply.sh Use `tune_multiply.sh` when you want to expand triplet specifications, retune a directory, or split a larger tuning job into parts.

Tune a compact Cartesian-product specification:

```
./tune_multiply.sh -t 300 4 10 15, 6 7 8, 23
```

Triplets are comma-separated groups. The command above expands to all `MxNxK` combinations with `M` in `4 10 15`, `N` in `6 7 8`, and `K=23`.

Tune an explicit list instead:

```
./tune_multiply.sh -t 300 1x1x1 2x2x2 13x5x7 23x23x23
```

Split the same work into four parts and run the second part:

```
./tune_multiply.sh -t 300 -j 4 -i 2 4 10 15, 6 7 8, 23
```

Parts also split a long job into sessions on a single device, where each session repeats the same command with the next part number:

```
PART=1  # next session: 2, then 3 and 4
./tune_multiply.sh -t 300 -j 4 -i ${PART} -p params/local 4 10 15, 6 7 8, 23
```

Keep the options that shape the list unchanged between sessions (`-r`, `-m`, `-n`, `-b`, `-f`, `-k`). With `-u`, do not add new shapes to the JSON directory between sessions.

Retune every JSON file found in a directory:

```
./tune_multiply.sh -u -p params/local -t 300
```

Retuning by age `--plan` lists the kernels of a JSON directory oldest first and writes that list as a file of triplets, or to standard output if no file is given. A kernel is dated by its youngest JSON file:

```
./tune_multiply.sh --plan retune.txt -p params/local -n 40
./tune_multiply.sh -f retune.txt -p params/local -t 300
```

The first command takes the 40 kernels that were tuned least recently (`-b` takes the most recent ones instead, and `-r`, `-m`, and `-n` limit the plan as they limit any other list). The second command works the plan. Keep the plan file between sessions rather than generating it again.

Each kernel that tuned successfully is marked `#done` in the plan and is skipped when the plan is used again, so an interrupted session resumes where it stopped and a kernel that failed stays in the plan. Only a generated plan is marked (a handwritten list is left alone), and only in a single session (`-j 1`). Delete the marks to tune the plan again:

```
sed "s/^#done //" retune.txt > retune.tmp && mv retune.tmp retune.txt
```

A plan lists every kernel of the directory. `-n` with `--plan` limits the plan, and `-n` with `-f` limits a single session, so repeating the same command works down the plan:

```
./tune_multiply.sh --plan retune.txt -p params/local
./tune_multiply.sh -f retune.txt -p params/local -n 2 -t 300
```

Once no unmarked line is left, generate the next plan:

```
grep -qv "^#" retune.txt || ./tune_multiply.sh --plan retune.txt -p params/local
./tune_multiply.sh -f retune.txt -p params/local -n 2 -t 300
```

Limit the generated work before splitting it:

```
./tune_multiply.sh -t 180 -r 4 32 -m 64 -n 100 \
  4 8 16 32, 4 8 16 32, 4 8 16 32
```

Useful options:

Option	Meaning
-t seconds	Time limit per kernel (default: 160)
-p path	Directory for JSON files
-s size	Benchmark batch size, also called stack size
-a level	Tuning level: 0=all ... 4=least tunables
-u	Retune JSON files found under `-p`
–plan [file]	Write kernels of `-p` by age, oldest first
-d	Ask the merge step to delete losing JSONs
–prefer new	Prefer newest duplicate (default: fastest)
-c	Continue with the next kernel after an error
-b	Reverse triplets, before `-n` and before parts
-j parts	Total number of tuning parts
-i index	Part to run, using 1-based numbering
-r low high	Keep kernels with low**3 < MNK <= high**3
-m extent	Keep kernels with M, N, and K no larger than this
-n count	Keep only the first count kernels (see `-b`)
-f file	Read MxNxK list from a file, resuming a plan
-k id	Use a predefined triplet set
-w seconds	Delay before tuning starts (default: 12, parts: 0)
-h	Print usage

With `-c` and without `-t`, there is no time limit per kernel. Option variables that are already set in the environment (e.g., `PART`, `NPARTS`, `JSONDIR`, `MAXTIME`, `CONTINUE`) act as if given.

Options the wrapper does not know are passed to `tune_multiply.py`. Such an option must carry its value as `--opt=value`, since a separate value would be read as the start of the triplet specification:

```
./tune_multiply.sh -t 300 --check=0 23x23x23
```

The tuning level defaults to `-1`, which fixes the same tunables as level 2. Levels 3 and 4 fix successively more of them.

Tuning from a File A text file with one `MxNxK` per line can drive the tuning session. Lines starting with `#` are comments, and inline comments after `#` are stripped. Whitespace within an entry is ignored:

```
./tune_multiply.sh -t 300 -f retune_shapes.txt -p params/local
```

Under MPI, the file entries are partitioned across ranks as usual:

```
mpirun -np 8 ./tune_multiply.sh -t 300 -f retune_shapes.txt -p params/local
```

Bulk Tuning with MPI Under MPI, the wrapper defaults `-j` to the MPI world size and `-i` to `rank + 1`. It also forwards a normalized local rank to `tune_multiply.py`, so the Python tuner can select a different device per local rank.

For most MPI launchers, this is enough:

```
mpirun -np 8 ./tune_multiply.sh -t 300 -p params/local \
  4 10 15, 6 7 8, 23
```

You can still spell out parts explicitly when the launcher or scheduler needs it:

```
mpirun \
  ./tune_multiply.sh -t 300 -j 4 -i 1 -p params/local \
    4 10 15, 6 7 8, 23 : \
  ./tune_multiply.sh -t 300 -j 4 -i 2 -p params/local \
    4 10 15, 6 7 8, 23 : \
  ./tune_multiply.sh -t 300 -j 4 -i 3 -p params/local \
    4 10 15, 6 7 8, 23 : \
  ./tune_multiply.sh -t 300 -j 4 -i 4 -p params/local \
    4 10 15, 6 7 8, 23 \
>out.log 2>&1
```

To retune an existing JSON directory across eight MPI ranks:

```
mpirun -np 8 ./tune_multiply.sh -u -p params/local -t 300
```

Managing Tuned Parameters JSON files are the working format (`params/*/` is not tracked), and CSV files are the deployable format. `smm_params.sh` converts between the two for every device at once, pairing a directory `params/<device>-0x<id>` with the CSV file `params/tune_multiply_<device>.csv` (double precision only):

```
./smm_params.sh             # merge every device
./smm_params.sh "H100*"     # merge matching devices only
./smm_params.sh -e          # the reverse: write JSONs to tmp/
```

The device id in the directory name must match the id of the merged parameters and that of the CSV file being replaced. Otherwise the merge reports both ids and writes nothing. A device without a CSV file needs `-f`.

A typical retune flow is:

```
make realclean
make WITH_GPU=P100
./tune_multiply.sh --plan retune.txt -p params/P100-0x2f6c
./tune_multiply.sh -f retune.txt -p params/P100-0x2f6c -t 300
./smm_params.sh P100
```

The last step also rebuilds the prediction model (see Tuned Parameters).

Keep GPU driver state persistent during tuning (e.g., `nvidia-smi -pm ENABLED` on headless NVIDIA systems).

Starting a New Device A new device can be seeded with the parameters of a similar one (e.g., the previous generation of the same vendor): `-e` expands its CSV file into JSON files:

```
./smm_params.sh -e P100
mv tmp/P100-0x2f6c params/NEW-0x0000
```

`-e` writes below `tmp/` only. The expanded files still name the device they came from. On the target machine, update the device name:

```
./tune_multiply.py -u -p params/NEW-0x0000
./smm_params.sh -f NEW
```

The merge reports the id it found. Rename the directory accordingly:

```
mv params/NEW-0x0000 params/NEW-0x1234
./smm_params.sh -f NEW
```

From there, retune with the normal flow.

Stencil

Finite-difference stencil computation for seismic wave propagation (RTM, FWI) on GPUs, with an 8th-order (radius-4) operator by default. Three kernel paths are available:

- **FP32** (default): the stencil in FP32 arithmetic, on any OpenCL device. Supports XYZ and ZYX memory layouts, PML absorbing boundaries, and the compact operator methods.
- **BF16-DPAS** (STENCIL_BF16=1): the stencil as BF16 matrix products on Intel matrix units (DPAS/XMX), accumulated in FP32.
- **INT8** (STENCIL_INT8=1): the stencil as INT8 matrix products (DPAS on Intel GPUs, dp4a on NVIDIA).

Build

```
make
```

Requires LIBXS in a sibling directory of LIBXSTREAM (`../../../libxs` from here) and an OpenCL runtime. The FP32 path (default) runs on any OpenCL device. The BF16 and INT8 paths need DPAS and cl_intel_subgroup_2d_block_io on Intel GPUs, the INT8 path also runs on NVIDIA (dp4a), and other devices fall back to scalar code (`STENCIL_BF16=2`, `STENCIL_INT8=2`), whose performance is not representative.

Build-time defines go into ECFLAGS, e.g., `make ECFLAGS="-DSTENCIL_PML_W=27"` (DFLAGS and CFLAGS are assembled by the build and a command line assignment would replace them):

- `STENCIL_PML_W`: absorbing layer width in cells (default: 20). A caller supplying its own eta sets its own damping width here; a larger value is safe, a smaller one leaves damped cells unattenuated.

```
make OCL=0
```

Compiles the FP32 kernel (kernels/stencil_fp32.cl) with the C compiler and runs it on the CPU instead of the GPU. Needs neither OpenCL nor the LIBXSTREAM library. The kernel is specialized at build time, so the work-group shape and optionally the grid extent are pinned by CPUDEF:

```
make OCL=0 CPUDEF="-DWG_X=128 -DWG_Y=8"
```

Define (CPUDEF)	Meaning
`-DWG_X=`, `-DWG_Y=`	work-group shape (default: full fast axis with capacity 1024, and 8)
`-DSTENCIL_NX=`, `-DSTENCIL_NY=`, `-DSTENCIL_NZ=`	pin the grid (all three or none); another grid is refused
`-DSTENCIL_LAYOUT=`	layout built in (0=XYZ, 1=blocked, 2=ZYX; default: 0)
`-DSTENCIL_PML=1`	PML built in (default: 0)
`-DSTENCIL_CPU_COMPACT=0`	drop the compact radii (8 instead of 32 instances)
`-DSTENCIL_PADDED=`	force the boundary treatment (0=clamp, 1=halo)
`-DSTENCIL_CPU_LANES=0`	1x1 work-groups instead of lane loops (default: 1); with `-DLIBXSTREAM_CPU_TEAM=1`, an OpenMP team per work-group

`STENCIL_BF16`, `STENCIL_INT8`, and `STENCIL_BLOCKED` are refused with OCL=0. Bind the threads, which is what `make OCL=0 test` does:

```
OMP_PROC_BIND=spread OMP_PLACES=cores ./stencil.x -n 512
```

The host build carries one kernel instance per combination of these environment variables and selects between them at run time:

Variable	Meaning
STENCIL_BF16S	wavefield storage limbs (1 = single BF16, 2 = two)
STENCIL_FP16S	IEEE FP16 wavefield storage (0/1)
STENCIL_HALO	halo padding size per axis
STENCIL_METHOD	operator method (0–3)
STENCIL_LAYOUT	accepted when it names the layout built in
STENCIL_PML	accepted when it matches the build

The default host build carries 32 instances. Drop the compact radii for a build of eight:

```
make OCL=0 CPUDEF="-DSTENCIL_CPU_COMPACT=0"
```

Asking for a combination the build does not carry is an error naming the flag to rebuild without, never a silent substitution. `STENCIL_PADDED` forces the boundary treatment and then compiles only that half.

The benchmark driver reaches the host build through `--exe`, which makes the CPU kernel sweepable like any device variant:

```
make OCL=0 BLDDIR=/tmp/b/obj OUTDIR=/tmp/b
./stencil.py --exe /tmp/b/stencil.x --kernel bf16s --sizes 256,512
```

FP16S is slow in the host build. BF16S=2 (`--kernel bf16s-split`) stores two limbs, i.e., as many bytes as FP32.

Usage

```
./stencil.x [options]

  -n <N>          grid dimension (NxNxN, default 256)
  -nx/ny/nz <N>   individual grid dimensions
  -t <steps>      time steps (default 100)
  -d <dims>       operator terms: 3=isotropic, 9=TTI (default 3, 9 needs STENCIL_BF16=1)
  -m <method>     operator method 0--3 (overrides STENCIL_METHOD)
  -i <init>       initial wavefield: rand | zero | gauss (default rand)
  -s <spacing>    grid spacing in meters (default 10.0), also --spacing
  -v <model>      velocity: const | grad | layered | <file.bin> (default grad)
  -vmin <vel>     minimum velocity m/s (default 1500)
  -vmax <vel>     maximum velocity m/s (default 4500)
  -w <steps>      warmup steps excluded from timing (default 5)
  -h, --help      show the options and exit

  -seg-salt       preset: SEG/EAGE Salt (676x676x210, h=20m)
  -overthrust     preset: SEG/EAGE Overthrust (801x801x187, h=25m)
```

A preset sets the grid, the spacing, the velocity range, and a synthetic velocity model; `-v` given after the preset replaces the model.

Performance is reported as GPoints/s (grid points updated per second):

```
GPoints/s = (nx x ny x nz x nsteps) / (time x 1e9)
```

`stencil.py` runs a sweep of `stencil.x` and writes CSV (optionally a plot); `./stencil.py --help` lists all options:

Option	Meaning
`--sizes`	grid sizes, comma lists, or ranges start:stop[:step] (default: 800)
`--kernel`	fp32 (default), fp32-split, bf16, int8, int8-split, bf16s, bf16s-split, fp16s
`--exe`	path to stencil.x (default: ./stencil.x)
`--output`, `--input`	CSV to write (default: stencil-.csv), or CSV to read instead of running
`--plot`	PNG output path
`--layout`	sets STENCIL_LAYOUT (0, 1, 2)
`--pml`	sets STENCIL_PML=1
`--check`	sets STENCIL_CHECK=1
`--ppw`, `--fit`, `--radius-fit`	set STENCIL_PPW, STENCIL_FIT, STENCIL_RADIUS_FIT

TTI mode With -d 9, the kernel adds the cross-derivative terms of tilted transverse isotropy. This requires the BF16-DPAS path (`STENCIL_BF16=1`, not with the ZYX layout); otherwise `-d 9` is refused with an error. The CPU check (`STENCIL_CHECK`) covers the isotropic terms only.

Velocity Models

The driver loads an external velocity model as a flat binary file (float32), one velocity volume without anisotropy parameters. The two 3D SEG/EAGE models below are publicly available.

SEG/EAGE Salt Body (3D)

- Grid: 676 x 676 x 210 (n1=210 depth, n2=n3=676 lateral)
- Spacing: 20 m
- Range: 1500–4482 m/s
- Size: ~385 MB (float32)
- Reference: Aminzadeh et al. (1997)

Downloads:

- Original (20 m): https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Salt/fvp

- Resampled (40 m, 115x338x338, for cheaper runs): https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Salt/BENCH_27PT/v.bin

SEG/EAGE Overthrust (3D)

- Grid: 801 x 801 x 187 (n1=187 depth, n2=n3=801 lateral)
- Spacing: 25 m
- Range: 2179–6000 m/s
- Size: ~450 MB (float32)
- Reference: Aminzadeh, Brac, and Kunz (1997)

Downloads:

- Original (25 m): https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Overthrust/fvp
- Resampled (50 m, 94x401x401): https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Overthrust/fvp.r50

Both models are hosted by the WIND project at Universite Cote d'Azur / Geoazur: https://www.geoazur.fr/WIND/bin/view/Main/Data/

Usage with this benchmark The binary format is flat float32, read without transposition into `-nx` x `-ny` x `-nz` with x fastest. The files above store depth (n1) fastest, hence depth becomes the x-axis. Velocities are in m/s; the driver squares them internally. Byte order is native (little-endian on x86). Download the velocity binary and run:

```
wget https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Salt/fvp
./stencil.x -seg-salt -nx 210 -ny 676 -nz 676 -v fvp -t 1000

wget https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Overthrust/fvp
./stencil.x -overthrust -nx 187 -ny 801 -nz 801 -v fvp -t 1000
```

Operator Methods

Methods (selected via STENCIL_METHOD environment variable) replace the radius-4 operator by a smaller radius r:

Method	Name	K	r	Description
0	direct	1	4	standard high-order (default)
1	compact-r1	4	1	compact tridiagonal runtime path
2	compact-r2	2	2	compact pentadiagonal runtime path
3	compact-fit	2	3	dispersion-fitted coefficients (r: `STENCIL_RADIUS_FIT`)

The compact-fit coefficients target a number of points per wavelength (`STENCIL_PPW`, default 8). The compact methods apply to the isotropic terms; TTI cross-terms use the direct operator.

Environment Variables

Variable	Meaning
STENCIL_BF16	BF16-DPAS kernel (1=native BF16, 2=FP32 arithmetic without DPAS, the fallback on devices without DPAS)
STENCIL_BF16S	BF16 wavefield storage limbs (0=off, 1=single 2-byte limb halves traffic, 2=two limbs FP32-equivalent)
STENCIL_FP16S	IEEE FP16 wavefield storage (0/1); single 2-byte value per point, halves traffic (10-bit mantissa, FP32 kernel)
STENCIL_INT8	INT8 kernel (1=DPAS on Intel or dp4a on NVIDIA, 2=scalar integer code, the fallback on other devices)
STENCIL_NDIGITS_A	operator digits of the BF16 and INT8 paths (1–3, default: 1)
STENCIL_I8_OP	INT8 operator: `implicit` (compact, default) or `banded` (radius-4 FD weights)
STENCIL_NDIGITS_X	INT8 wavefield digits (1–3, default: 2 with the implicit operator, else 3)
STENCIL_R_GATHER	INT8 gather radius (1–4, default: 2 with the implicit operator, else 4)
STENCIL_METHOD	operator method (0–3, default 0)
STENCIL_PPW	compact-fit points per wavelength (default: 8)
STENCIL_FIT	compact-fit method (0=uniform L2, 1=Ricker-weighted, 2=minimax; default: 2)
STENCIL_RADIUS_FIT	compact-fit radius (2 or 3, default: 3)
STENCIL_STRIPS_PER_WG	adjacent N-strips handled by one work-group (default: 2, INT8: 1; must divide 64)
STENCIL_SG	subgroup size override (default: device preferred)
STENCIL_GRF256	force 256-GRF mode (0/1, default: 0)
STENCIL_TRIM	drop least-significant digit products (BF16 and INT8 paths, default: 1)
STENCIL_LU	loop unroll strategy (-1=none, 0=inner, 1=outer; default: 0)
STENCIL_LAYOUT	memory layout (0=XYZ, 1=blocked, 2=ZYX; default: 0); with ZYX, `STENCIL_BF16` falls back to the FP32 kernel
STENCIL_BLOCKED	1 selects layout 1 (blocked) when

STENCIL_LAYOUT is not set | | STENCIL_HALO | halo/padding size per axis (default: 0) | | STENCIL_PML | enable PML absorbing boundary (0/1, default: 0) | | STENCIL_PML_ETA | PML damping peak (default: 0.0199) | | STENCIL_FP32_WG_X, STENCIL_FP32_WG_Y | FP32 direct-kernel work-group shape (default: 32x8); powers of two with X <= 64, Y <= 32, and X*Y = 256, else the default | | STENCIL_FP32_SBLOCK | FP32 direct-kernel slow-axis SLM microblock (1 or 2, default: 2 except ZYX+PML) | | STENCIL_FP32_BLOCK_IO | enable FP32 Intel 2D block reads for padded 32x8 cases (0/1, default: 0); needs `STENCIL_FP32_SBLOCK=1` | | STENCIL_CHECK | unset: print output statistics; 1: also compare against a CPU reference; 0: neither | | STENCIL_HINT | debug: non-zero allocates device buffers with the atomics hint (default: 0) | | STENCIL_TRACE | debug: if set, trace the driver phases to stderr | | STENCIL_DUMP_OP | debug: if set, print the reconstruction error of the BF16 operator |

Integration

The stencil API (stencil_opencl.h) accepts device pointers for all buffer arguments (wavefield, output, velocity). No host-device transfers happen inside the dispatch path.

```
stencil_context_t ctx;
stencil_init(&ctx, verbosity, method);  /* method < 0: STENCIL_METHOD */
ctx.nterms = nterms;                    /* 3 (default) or 9 */
stencil_configure(&ctx, nx, ny, nz);
stencil_precompute_operators(&ctx, fd_weights, STENCIL_RADIUS);
stencil_seed_exp_buf(&ctx, p_host, nx, ny, nz);  /* INT8 path only */
for (t = 0; t < nsteps; ++t) {          /* p_old receives the next step */
  stencil_apply_laplacian(&ctx, p_cur, p_old, vel, dt * dt, h, nterms);
  tmp = p_cur; p_cur = p_old; p_old = tmp;
}
libxstream_stream_sync(ctx.stream);
stencil_finalize(&ctx);
```

`fd_weights` holds the 2*STENCIL_RADIUS+1 second-derivative weights for the grid spacing h (center at index STENCIL_RADIUS), and `vel` holds the squared velocity. Launches are asynchronous on `ctx.stream`. A caller with its own absorbing boundary (STENCIL_PML=1) sets the device buffers `ctx.eta` and `ctx.phi` between `stencil_init` and `stencil_configure`.

Buffers are expected in the configured layout and storage format. These helpers convert from an XYZ float32 array on the host:

| Helper | Converts to | ||| | `stencil_pack_blocked` | blocked layout (size: `stencil_blocked_size`) | | `stencil_pack_bf16s` | BF16S/FP16S storage (ndigits: 0 = FP16, 1 or 2 = BF16 limbs) | | `stencil_pack_bf16s_blocked` | BF16S/FP16S storage in the blocked layout | | `stencil_pack_bf16s_zyx` | BF16S/FP16S storage in the ZYX layout with halo | | `stencil_unpack_bf16s` | back from BF16S/FP16S storage to float32 |

Initialization and USM control LIBXSTREAM provides libxstream_init_config for explicit control over the memory model before initialization:

```
libxstream_init_config_t cfg;
libxstream_init_config_default(&cfg);  /* all fields = -1 (default) */
cfg.usm = 1;      /* force Intel USM extensions */
cfg.device = 0;   /* select device index */
libxstream_init_config(&cfg);
```

USM levels (cfg.usm):

| Level | Meaning | ||| | -1 | env/default (LIBXSTREAM_USM): level 1 on an Intel discrete GPU unless Intel MPI runs without offload support, disabled on Intel GPUs with unified memory, else as level 2 | | 0 | disable USM, force clCreateBuffer path | | 1 | Intel USM extensions (clDeviceMemAllocINTEL) | | 2 | OpenCL 2.0 SVM coarse-grain only | | 3 | OpenCL 2.0 SVM with device-reported capabilities |

When USM is active (level 1), device pointers from any source (SYCL, Level Zero, raw clDeviceMemAllocINTEL) can be passed to the stencil API without conversion.

SYCL interoperability SYCL USM device allocations (sycl::malloc_device) can be passed directly to stencil_apply_laplacian on Intel GPUs. Requirements:

- Initialize LIBXSTREAM with cfg.usm = 1, or set the environment variable LIBXSTREAM_USM=1.
- The SYCL queue and the libxstream OpenCL context must target the same physical device (shared Level Zero context).
- Synchronization: use sycl::queue::ext_oneapi_submit_barrier() before calling stencil_apply_laplacian, and libxstream_stream_sync after, to order work correctly.
- No format conversion needed: float32 USM buffers are used as-is (velocity is expected as v^2).

External OpenCL consumers Any OpenCL application can call the API by passing cl_mem-backed device pointers obtained via libxstream_mem_allocate, or USM pointers from clDeviceMemAllocINTEL directly.

File Layout

```
samples/stencil/
  Makefile              build rules
  README.md             this file
  stencil.c             host driver (benchmark, model loading)
  stencil.py            Python benchmark harness (CSV/plot output)
  stencil_opencl.c      OpenCL context, kernel dispatch, exp_buf management
  stencil_opencl.h      public API, compile-time parameters
  stencil_kernels.h     generated at build time from .cl sources
  stencil_pack.c        layout and storage conversion helpers
  stencil_pml.c/.h      absorbing boundary setup
  stencil_weights.c/.h  operator weights of the methods
  stencil_cpu.c         host build (OCL=0): kernel instances and launcher
  stencil_cpu_set.h     host build: list of kernel instances
  stencil_cpu_instance.h  host build: one kernel instance
  stencil_hostmem.h     host build: stand-in for the LIBXSTREAM memory API
  kernels/
    stencil_common.cl   parameters, gather macros, layout indexing
    stencil_fp32.cl     FP32 path: stencil_apply_direct (default)
    stencil_bf16.cl     BF16 path: stencil_apply, stencil_apply_tti
    stencil_int8.cl     INT8 path: stencil_apply_int8

libxstream/opencl/
  libxstream_common.h   IEEE utilities, BF16 conversion, EXP2I, unroll macros
  libxstream_ozaki.h    Dekker split, BF16 DPAS macros
  libxstream_cpu_begin.h, libxstream_cpu_end.h  host translation of a kernel (OCL=0)
```